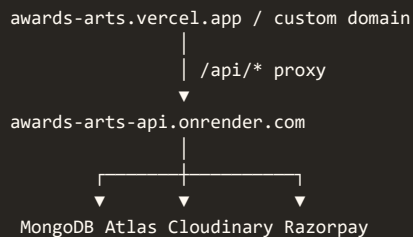


Awards Arts: Vercel + Render deployment guide

Tumhare project ke liye final architecture:



- Vercel: React/Vite frontend
- Render: Node.js/Express backend
- MongoDB Atlas: database
- Cloudinary: product images/uploads
- Razorpay: payments and webhooks
- GitHub: source code and automatic deployments

Vite officially Vercel-supported hai, aur Render Docker-based Node services support karta hai. [Vercel Vite guide](#), [Render web-service guide](#)

Phase 1: Deployment se pehle local verification

Step 1: Project folder mein terminal kholo

PowerShell mein:

```
cd "D:\Delta Trophies\LegacyTrophies\Legacy-Awards"
```

`cd` ka matlab "change directory" hai. Isse terminal tumhare project ke root folder par aa jayega.

Step 2: Git status check karo

```
git status
```

Purpose:

- Confirm karega ki koi uncommitted file pending toh nahi.
- Deployment mein sirf committed/pushed code jayega.
- Ideal output: `working tree clean`.

Step 3: Exact dependencies install karo

```
npm ci
```

Purpose:

- `package-lock.json` ke exact versions install karta hai.
- Local aur production dependencies same rehengi.
- `npm install` ke comparison mein deployment verification ke liye zyada deterministic hai.

Note: `npm ci` existing `node_modules` recreate kar sakta hai. Source code/database delete nahi hoti.

Step 4: Security audit dekho

```
npm audit
```

Purpose:

- Dependencies mein known vulnerabilities report karta hai.
- Project mein pehle audit ne 2 **moderate** aur 1 **high** report ki thi, isliye production se pehle result review karna zaroori hai.

Blindly yeh mat chalana:

```
npm audit fix --force
```

--force breaking dependency upgrades kar sakta hai. Pehle report dekhkar controlled fix karna chahiye.

Step 5: Backend tests chalao

```
npm test
```

Purpose:

- Authentication
- Admin/sales permissions
- Quote workflow
- Payment verification
- Manual payments
- API validation

Expected result:

```
13 test files passed
53 tests passed
```

Step 6: Production frontend build test karo

```
npm run build
```

Purpose:

- React/Vite ka production bundle generate karta hai.
- JSX/import/build errors deployment se pehle mil jayenge.
- Output **client/dist** mein generate hoga.

Expected end:

```
built in ...
```

Phase 2: GitHub repository prepare karna

Step 1: GitHub remote verify karo

```
git remote -v
```

Expected:

```
origin https://github.com/USERNAME/REPOSITORY.git
```

Agar remote configured nahi hai:

```
git remote add origin https://github.com/USERNAME/REPOSITORY.git
```

Purpose:

- Local repository ko GitHub repository se connect karta hai.

Step 2: Current branch dekho

```
git branch --show-current
```

Normally output:

```
main
```

Step 3: Changes commit karo

Agar changes pending hain:

```
git add --all
git commit -m "Prepare Awards Arts for production deployment"
```

Explanation:

- `git add --all`: modified/new/deleted files staging mein rakhta hai.
- `git commit`: staged changes ka permanent Git snapshot banata hai.

Step 4: GitHub par push karo

```
git push origin main
```

Purpose:

- Local `main` branch GitHub par upload karta hai.
- Render aur Vercel isi GitHub repository se deployment karenge.

Important: `.env`, API secret, JWT secret, MongoDB password ya Razorpay secret GitHub par commit mat karna.

Phase 3: Existing deployment configuration update karna

Tumhari repository mein already:

- `render.yaml`
- `Dockerfile`
- `client/vercel.json`

available hain. Lekin config mein purana `legacy-awards` service URL abhi present ho sakta hai.

Step 1: `render.yaml` update karo

`render.yaml` mein service block approximately aisa rakho:

```
services:
- type: web
  name: awards-arts-api
  env: docker
  plan: free
  autoDeploy: true
  dockerfilePath: ./Dockerfile
  healthCheckPath: /api/health
  envVars:
    - key: NODE_ENV
      value: production
    - key: APP_ORIGIN
      value: https://awards-arts.vercel.app
    - key: COOKIE_SECURE
      value: "true"
    - key: JWT_ACCESS_TTL
      value: 15m
    - key: JWT_REFRESH_TTL
      value: 7d
    - key: LOG_LEVEL
      value: info
    - key: MONGODB_URI
      sync: false
```

```
- key: JWT_ACCESS_SECRET
  sync: false
- key: JWT_REFRESH_SECRET
  sync: false
- key: CLOUDINARY_CLOUD_NAME
  sync: false
- key: CLOUDINARY_API_KEY
  sync: false
- key: CLOUDINARY_API_SECRET
  sync: false
- key: RAZORPAY_KEY_ID
  sync: false
- key: RAZORPAY_KEY_SECRET
  sync: false
- key: RAZORPAY_WEBHOOK_SECRET
  sync: false
```

`sync: false` ka matlab secret GitHub file mein store nahi hoga. Blueprint create karte waqt Render uski value maangega. [Render Blueprint secrets](#)

Plan selection

Initial testing:

```
plan: free
```

Real customer/payment production:

```
plan: starter
```

Ya Render dashboard se paid always-on plan select karo.

Free Render service 15 minutes inactivity ke baad sleep ho sakti hai. First request ko cold start lag sakta hai, isliye real payments/webhooks ke liye always-on paid instance recommended hai. [Render free limitations](#)

Step 2: Config commit/push karo

```
git add render.yaml
git commit -m "Configure Awards Arts Render service"
git push origin main
```

Phase 4: Secure JWT secrets generate karna

Do different secrets generate karo.

PowerShell:

```
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
```

Command ko do baar run karo.

Purpose:

- `randomBytes(48)` cryptographically secure 48 random bytes generate karta hai.
- `base64url` unhe environment-variable-safe text mein convert karta hai.
- Pehli value `JWT_ACCESS_SECRET`.
- Doosri, different value `JWT_REFRESH_SECRET`.

Rules:

- Dono secrets identical nahi hone chahiye.
- GitHub/code/WhatsApp par paste mat karna.
- Sirf Render Environment section mein save karna.
- Production mein local development secrets reuse mat karna.

Webhook secret ke liye ek third value generate karo:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Is value ko:

- Render mein `RAZORPAY_WEBHOOK_SECRET`
- Razorpay webhook setup mein `Secret`

dono jagah exactly same dalna hai.

Phase 5: Render backend deploy karna

Step 1: Render dashboard open karo

[Render Dashboard](#) par login karo.

Step 2: Blueprint create karo

1. New button click karo.
2. Blueprint select karo.
3. GitHub repository connect karo.
4. Awards Arts repository select karo.
5. Blueprint file path:

```
render.yaml
```

6. Blueprint name:

```
awards-arts
```

Render root `render.yaml` automatically detect kar sakta hai. Blueprint secrets with `sync: false` initial setup par prompt hote hain. [Render Blueprints](#)

Step 3: Environment secrets enter karo

Required variables

`MONGODB_URI`

Example structure:

```
mongodb+srv://USERNAME:PASSWORD@CLUSTER.mongodb.net/awardsarts?retryWrites=true&w=majority
```

Actual credentials hi use karna.

Agar MongoDB password mein special characters hain—jaise `@`, `/`, `:`, `#`, `%`—to password URL-encoded hona chahiye.

`JWT_ACCESS_SECRET`

Pehla generated random secret.

`JWT_REFRESH_SECRET`

Doosra generated random secret.

Cloudinary

```
CLOUDINARY_CLOUD_NAME  
CLOUDINARY_API_KEY  
CLOUDINARY_API_SECRET
```

Cloudinary dashboard ke exact new-account credentials dalna.

Razorpay

Testing phase:

```
RAZORPAY_KEY_ID=rzp_test_...
RAZORPAY_KEY_SECRET=...
RAZORPAY_WEBHOOK_SECRET=<generated-webhook-secret>
```

Test keys aur live keys mix mat karna.

Step 4: Deploy start karo

Apply/Create Blueprint click karo.

Render:

1. Docker image build karega.
2. Node 22 install karega.
3. Dependencies install karega.
4. React frontend build karega.
5. Express backend start karega.
6. `/api/health` ko health check ke liye call karega.

Tumhara Docker container non-root `node` user se run hota hai, jo security ke liye correct hai.

Step 5: Render logs check karo

Expected log lines:

```
MongoDB connected
API listening
```

Agar MongoDB allowlist configured nahi hai, API degraded mode mein start ho sakti hai. Health endpoint phir bhi respond kar sakta hai, isliye JSON ke andar database status bhi check karna hai.

Step 6: Backend URL note karo

Example:

```
https://awards-arts-api.onrender.com
```

Exact URL Render dashboard se copy karo. Guess mat karna.

Step 7: Health endpoint test karo

Browser:

```
https://awards-arts-api.onrender.com/api/health
```

PowerShell:

```
Invoke-RestMethod "https://awards-arts-api.onrender.com/api/health" |
  ConvertTo-Json -Depth 5
```

Purpose:

- `Invoke-RestMethod`: API ko HTTP request bhejta hai.
- `ConvertTo-Json -Depth 5`: nested response readable format mein show karta hai.

Expected:

```
{
  "success": true,
  "data": {
    "status": "ok",
    "database": {
      "connected": true
    }
  }
}
```

Sirf HTTP 200 sufficient nahi hai; `database.connected` bhi `true` hona chahiye.

Phase 6: MongoDB Atlas ko Render se securely connect karna

MongoDB Atlas sirf allowlisted IPs se connections accept karta hai. [Atlas IP access list](#)

Step 1: Render outbound IP ranges nikalo

1. Render service `awards-arts-api` kholo.
2. Top-right Connect click karo.
3. Outbound tab kholo.
4. Saare CIDR ranges copy karo.

Render service region ke specific shared outbound ranges deta hai. Service unmein se kisi range ka IP use kar sakti hai, isliye saare listed ranges add karna zaroori hai. [Render outbound IP documentation](#)

Step 2: Atlas Network Access update karo

1. MongoDB Atlas kholo.
2. Correct project select karo.
3. Security → Network Access.
4. Add IP Address.
5. Render ke har CIDR range ko add karo.
6. Description:

```
Awards Arts Render Singapore
```

Region ke according naam change kar sakte ho.

7. Confirm/save karo.

Production mein permanently `0.0.0.0/0` avoid karo. Iska matlab entire internet allowed hota hai.

Step 3: Database user permissions check karo

Atlas:

1. Security → Database Access
2. Production application user kholo.
3. Minimum required access do:

```
Read and write to specific database
```

4. Database:

```
awardsarts
```

Application ko Atlas admin/project owner permissions dene ki zarurat nahi.

Step 4: Render redeploy

Render dashboard:

```
Manual Deploy → Deploy latest commit
```

Phir health endpoint dobara check karo.

Phase 7: Vercel proxy ko actual Render URL dena

`client/vercel.json` open karo.

Is section ko actual backend URL se update karo:

```
{
  "rewrites": [
    {
      "source": "/api/:path*",
      "destination": "https://awards-arts-api.onrender.com/api/:path*"
    },
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

Actual Render URL use karo.

Why proxy?

Frontend browser mein request karega:

```
https://awards-arts.vercel.app/api/v1/products
```

Vercel internally forward karega:

```
https://awards-arts-api.onrender.com/api/v1/products
```

Benefits:

- Frontend aur cookies same website origin par behave karte hain.
- Browser cross-site cookie problems reduce hote hain.
- Public frontend code ko direct API-base environment variable nahi chahiye.

`VITE_API_URL` production mein blank/unset rakho when using this proxy.

Config push karo:

```
git add client/vercel.json
git commit -m "Point frontend API proxy to Awards Arts backend"
git push origin main
```

Phase 8: Vercel frontend deploy karna

Step 1: Vercel dashboard kholo

[Vercel Dashboard](#) par login karo.

Step 2: GitHub repository import karo

1. Add New → Project
2. GitHub repository select karo.
3. Import click karo.

Step 3: Project settings configure karo

Project name

```
awards-arts
```

Agar unavailable ho:

```
awards-arts-india
```

Framework preset

```
Vite
```

Root Directory

```
client
```

Vercel monorepo deployments mein specific project directory select karna officially supported hai. [Vercel monorepo guide](#)

Build Command

```
npm run build
```

Purpose:

- Vite production bundle generate karta hai.

Output Directory

```
dist
```

Purpose:

- Vite compiled HTML/CSS/JS isi folder mein output karta hai.

Install Command

Default rehne do, ya explicitly:

```
npm install
```

Project npm workspace use karta hai. Root `package-lock.json` ko include karne ke liye Vercel ke Root Directory section mein Include source files outside of the Root Directory in the Build Step enabled rehna chahiye.

Step 4: Vercel environment variables

Production mein ye variables add mat karna:

```
VITE_DEV_ADMIN_EMAIL  
VITE_DEV_ADMIN_PASSWORD  
VITE_DEV_SALES_1_EMAIL  
VITE_DEV_SALES_1_PASSWORD  
VITE_DEV_SALES_2_EMAIL  
VITE_DEV_SALES_2_PASSWORD  
VITE_DEV_SALES_3_EMAIL  
VITE_DEV_SALES_3_PASSWORD  
VITE_DEV_CUSTOMER_EMAIL  
VITE_DEV_CUSTOMER_PASSWORD
```

Development autofill buttons production build mein `import.meta.env.DEV` ke through disabled hain, lekin production Vercel environment mein credentials rakhne ka koi reason nahi.

`VITE_API_URL` bhi unset rakho because `/api` rewrite proxy use ho raha hai.

Step 5: Deploy click karo

Vercel build complete hone ke baad URL milega:

```
https://awards-arts.vercel.app
```

Agar different URL mila, exact URL copy karo.

Phase 9: Render APP_ORIGIN final karna

Render dashboard:

1. `awards-arts-api` service kholo.
2. Environment open karo.
3. `APP_ORIGIN` update karo.

Single production URL:

```
https://awards-arts.vercel.app
```

Agar custom domain bhi hai:

```
https://awards-arts.vercel.app,https://awardsarts.in,https://www.awardsarts.in
```

Rules:

- Comma-separated values.
- Trailing slash mat lagana.
- `APP_ORIGIN` frontend URL hai, Render API URL nahi.

Correct:

```
https://awards-arts.vercel.app
```

Incorrect:

```
https://awards-arts.vercel.app/  
https://awards-arts-api.onrender.com
```

Select:

Save and deploy

Render environment changes previous deployment automatically modify nahi karte; service redeploy required hota hai. [Render environment-variable guide](#)

Phase 10: Razorpay test webhook configure karna

Backend public hone ke baad webhook URL:

```
https://awards-arts-api.onrender.com/api/v1/payments/webhook
```

Vercel URL webhook ke liye use mat karo. Direct Render backend use karo.

Razorpay Dashboard

1. Test Mode select karo.
2. Accounts & Settings
3. Webhooks
4. Add New Webhook

Webhook URL

```
https://awards-arts-api.onrender.com/api/v1/payments/webhook
```

Secret

Exactly wahi value jo Render mein:

```
RAZORPAY_WEBHOOK_SECRET
```

Alert email

Tumhara monitored business email.

Events

Select:

```
payment.authorized  
payment.captured  
payment.failed  
payment.refunded
```

Webhook signature verification mandatory hai. Tumhara backend raw request body aur `x-razorpay-signature` verify karta hai. Razorpay immediate success ke saath server verification/webhooks use karne ki recommendation deta hai. [Razorpay payment verification](#)

Auto capture

Razorpay Dashboard mein automatic payment capture enable karo. Successful business order sirf verified/captured payment ke baad paid mark hona chahiye.

Phase 11: Existing production admin

Tumne admin current Atlas database mein already create kiya hua hai. Same `MONGODB_URI` Render par use karoge toh admin automatically available hoga.

Production mein yeh commands mat chalana:

```
npm run demo:sales
npm run demo:workflow:reset
npm run seed
```

Reason:

- Demo workflow reset quotations/orders/payments modify/delete kar sakta hai.
- Production data par destructive ho sakta hai.
- Demo sales accounts production mode mein login se blocked hain, but production database mein unnecessary demo data nahi hona chahiye.

Phase 12: Complete production verification

1. Frontend check

Open:

```
https://awards-arts.vercel.app
```

Verify:

- Awards Arts name
- Navbar/footer
- Home page
- Categories
- Product listing
- Product detail images

2. API proxy check

Open:

```
https://awards-arts.vercel.app/api/health
```

Is request ko Vercel Render par forward karega.

3. Product API check

```
https://awards-arts.vercel.app/api/v1/products?limit=24
```

Expected:

- 200

- Products returned
- Total approximately 625

4. Categories check

<https://awards-arts.vercel.app/api/v1/categories>

Expected:

- 13 active categories

5. Admin login

<https://awards-arts.vercel.app/admin/login>

Verify:

- Development autofill button absent
- Correct admin login works
- Products/categories/orders visible

6. Sales login

Verify:

- Development Sales 1/2/3 buttons absent
- Real active sales account works
- Assigned leads only show
- New lead badge updates

7. Customer workflow

Test:

1. Product open karo.
2. Quantity select karo.
3. Quote cart mein add karo.
4. Customer details submit karo.
5. Admin mein quote visible ho.
6. Sales/admin assignment test karo.
7. Final quote send karo.
8. Customer accept kare.
9. Admin online payment select kare.
10. Razorpay test payment complete karo.
11. Customer page paid state show kare.
12. Admin paid-orders tab mein order show ho.
13. Refresh ke baad state persist honi chahiye.

8. Cloudinary upload/delete

Admin se test image:

1. Upload karo.
2. Product save karo.
3. Image website par verify karo.
4. Image remove karo.
5. Cloudinary Media Library mein deletion verify karo.

9. Razorpay webhook test

Razorpay dashboard mein webhook delivery dekho:

- HTTP response expected: **204**
- Failed delivery nahi honi chahiye
- Wrong signature request reject honi chahiye
- Repeated event duplicate paid order create nahi karna chahiye

Phase 13: Custom domain add karna

Vercel:

1. Project kholo.
2. Settings → Domains
3. Add:

```
awardsarts.in
```

4. Add:

```
www.awardsarts.in
```

5. Vercel jo DNS records deta hai, domain provider par exactly add karo.

Vercel apex domain ke saath **www** configuration suggest karta hai. [Vercel custom domains](#)

Domain active hone ke baad Render **APP_ORIGIN**:

```
https://awardsarts.in,https://www.awardsarts.in
```

Agar Vercel URL bhi allow rakhna hai:

```
https://awards-arts.vercel.app,https://awardsarts.in,https://www.awardsarts.in
```

Render redeploy karo.

Razorpay website/domain settings mein bhi final domain register/update karo.

Common errors

Vercel page refresh par 404

Cause:

- `client/vercel.json` detect nahi hua.
- Root directory galat hai.

Fix:

```
Root Directory = client
```

Aur second rewrite present ho:

```
{
  "source": "/*",
  "destination": "/index.html"
}
```

/api par 404/502

Check:

- `client/vercel.json` destination actual Render URL hai.

- URL mein duplicate `/api/api` nahi hai.
- Render health endpoint working hai.

CORS error

Check `APP_ORIGIN`:

```
https://actual-vercel-url.vercel.app
```

Trailing slash remove karo, Render redeploy karo.

Login successful but refresh ke baad logout

Check:

```
NODE_ENV=production
COOKIE_SECURE=true
```

Frontend requests `/api` Vercel proxy se ja rahi honi chahiye.

Render deploy port error

Render automatically `PORT` variable provide karta hai. Tumhara server already `process.env.PORT` use karta hai. Manually `PORT=5000` set karne ki zarurat nahi.

Render public services ko supplied `PORT` par bind hona chahiye. [Render port binding](#)

Health API works but products nahi aate

Health response ke andar check karo:

```
"database": {
  "connected": true
}
```

Agar false:

- MongoDB URI incorrect
- Password URL-encoded nahi
- Render outbound ranges Atlas allowlist mein missing
- Wrong database user permissions

Razorpay webhook 503

Cause:

```
RAZORPAY_WEBHOOK_SECRET
```

missing hai.

Razorpay webhook signature error

Render aur Razorpay mein webhook secret character-for-character same hona chahiye.

`RAZORPAY_KEY_SECRET` ko webhook secret mat use karo. Dono separate secrets hain.

Final production environment checklist

Render:

```
NODE_ENV=production
APP_ORIGIN=https://your-final-frontend-domain
COOKIE_SECURE=true
JWT_ACCESS_TTL=15m
JWT_REFRESH_TTL=7d
```

```
LOG_LEVEL=info
MONGODB_URI=<secret>
JWT_ACCESS_SECRET=<unique-secret>
JWT_REFRESH_SECRET=<different-secret>
CLOUDINARY_CLOUD_NAME=<secret>
CLOUDINARY_API_KEY=<secret>
CLOUDINARY_API_SECRET=<secret>
RAZORPAY_KEY_ID=<test-or-live-key>
RAZORPAY_KEY_SECRET=<matching-test-or-live-secret>
RAZORPAY_WEBHOOK_SECRET=<separate-webhook-secret>
```

Vercel:

```
No backend secrets
No MongoDB URI
No Cloudinary API secret
No Razorpay key secret
No development login credentials
VITE_API_URL unset when proxy is used
```

Deployment order strictly yeh rakho:

```
GitHub push
→ Render backend
→ MongoDB allowlist
→ Backend health verification
→ vercel.json Render URL update
→ Vercel frontend
→ Render APP_ORIGIN update
→ Razorpay test webhook
→ Complete test payment
→ Custom domain
→ Live Razorpay keys/webhook
```